

“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD PERUANA LOS ANDES



FACULTAD INGENIERIA
ESPECIALIDAD: SISTEMAS Y COMPUTACION

Consultas Basicas a Nivel de Fila con SQL Server - Desarrollo 30
Proyectos

CURSO:

Base de Datos II

DOCENTE:

-Fernández Bejarano Raúl Enrique

INTEGRANTES:

- Rosales Crispín Aristóteles Onassis

Ciclo:

5°SEMESTRE

HUANCAYO- JUNIN
2025

INFORME DE LOS 30 EJERCICIOS SQL

Base de datos: GestionInventarioTecnologico

Introducción:

Este documento presenta el desarrollo de los 30 ejercicios SQL contenidos en el archivo proporcionado. Cada ejercicio se organiza con la misma estructura para facilitar su revisión académica: **enunciado, código SQL, análisis y evidencia**. El trabajo se centra en consultas básicas y de nivel intermedio sobre una base de datos de inventario tecnológico.

Desarrollo:

PROYECTO 1: Mostrar todos los registros de la tabla equipo

Enunciado: Mostrar todos los registros de la tabla Equipo_Inventario.

Código SQL:

```
SELECT * FROM Equipo_Inventario;
```

Análisis: Esta consulta recupera toda la información almacenada en la tabla central del inventario. Se utiliza cuando se necesita revisar el contenido completo sin aplicar filtros.

Evidencia: Resultado con todos los campos y registros de Equipo_Inventario en SQL Server.

```

-- =====
-- Proyecto 1: Mostrar todos los registros de la tabla equipo
-- =====
-- Enunciado: Mostrar todos los registros de la tabla equipo
-- Verificación y Resultado:
SELECT * FROM Equipo_Inventario;
GO

-- =====
-- Proyecto 2 - Mostrar columnas específicas de usuarios
-- =====
-- Enunciado: Mostrar únicamente el código y nombre de los usuarios

```

Id_Inventario	Id_CodigoAlmacen	ID_Hardware	ID_Software	ID_General	ID_Partes	Codigo_Anidesk
INV001	ALM001	HW001	SW001	GEN001	AP001	1001
INV002	ALM001	HW002	SW002	GEN002	AP002	1002
INV003	ALM001	HW003	SW003	GEN003	AP003	1003
INV004	ALM001	HW004	SW004	GEN004	AP004	1004
INV005	ALM001	HW005	SW005	GEN005	AP005	1005
INV006	ALM001	HW006	SW006	GEN006	AP006	1006
INV007	ALM001	HW007	SW007	GEN007	AP007	1007
INV008	ALM001	HW008	SW008	GEN008	AP008	1008
INV009	ALM001	HW009	SW009	GEN009	AP009	1009
INV010	ALM001	HW010	SW010	GEN010	AP010	1010
INV011	ALM001	HW011	SW011	GEN011	AP011	1011

PROYECTO 2: *Mostrar columnas específicas de usuarios*

Enunciado: Mostrar únicamente el código y nombre de los usuarios.

Código SQL:

```
SELECT Id_CodigoAlmacen, Usuario
FROM Ficha_Inventario_Tecnologico;
```

Análisis: La consulta selecciona solo los datos necesarios, lo que mejora la claridad del resultado y evita mostrar columnas innecesarias.

Evidencia: Lista con Id_CodigoAlmacen y Usuario.

```
-- =====
-- Proyecto 2 - Mostrar columnas específicas de usuarios
-- =====
-- Enunciado: Mostrar únicamente el código y nombre de los usuarios.
-- Verificación:
--
-- Consulta del ejercicio:
SELECT * FROM Ficha_Inventario_Tecnologico;
SELECT Id_CodigoAlmacen, Usuario
FROM Ficha_Inventario_Tecnologico;
GO
-- =====
```

121 %

Results Messages

	Id_CodigoAlmacen	Dependencia	Fecha	Cod_Tecnologia	Usuario
1	ALM001	Gerencia General	2026-05-26	TI	Juan Pérez

	Id_CodigoAlmacen	Usuario
1	ALM001	Juan Pérez

PROYECTO 3: Uso de alias para columnas

Enunciado: Mostrar el nombre del usuario utilizando alias personalizados.

Código SQL:

```
SELECT Usuario AS Nombre_Usuario_Responsable
FROM Ficha_Inventario_Tecnologico;
```

Análisis: El alias cambia el nombre visual de la columna en el resultado, haciendo la consulta más legible y presentable.

Evidencia: Columna mostrada con el encabezado Nombre_Usuario_Responsable.

```
-- =====
-- Proyecto 3 - Uso de alias para columnas
-- =====
-- Enunciado: Mostrar el nombre del usuario utilizando alias personalizados.
-- Verificación:
-- Consulta del ejercicio:
SELECT * FROM Ficha_Inventario_Tecnologico;
SELECT Usuario AS Nombre_Usuario_Responsable
FROM Ficha_Inventario_Tecnologico;
GO
-- =====
```

121 %

Results Messages

	Id_CodigoAlmacen	Dependencia	Fecha	Cod_Tecnologia	Usuario
1	ALM001	Gerencia General	2026-05-26	TI	Juan Pérez

	Nombre_Usuario_Responsable
1	Juan Pérez

PROYECTO 4: Uso de alias para tablas

Enunciado: Mostrar datos de la tabla dependencia utilizando alias.

Código SQL:

```
SELECT f.Id_CodigoAlmacen, f.Dependencia
FROM Ficha_Inventario_Tecnologico AS f;
```

Análisis: El alias f simplifica la escritura de la consulta y facilita su lectura cuando se trabaja con varias tablas.

Evidencia: Resultado con los campos solicitados de Ficha_Inventario_Tecnologico.

```
-- =====
-- Proyecto 4 - Uso de alias para tablas
-- =====
-- Enunciado: Mostrar datos de la tabla dependencia utilizando
-- Verificación:
SELECT * FROM Ficha_Inventario_Tecnologico;
-- Consulta del ejercicio:
SELECT f.Id_CodigoAlmacen, f.Dependencia
FROM Ficha_Inventario_Tecnologico AS f;
GO
-- =====
```

121 %

Results Messages

	Id_CodigoAlmacen	Dependencia	Fecha	Cod_Tecnologia	Usuario
1	ALM001	Gerencia General	2026-05-26	TI	Juan Pérez

	Id_CodigoAlmacen	Dependencia
1	ALM001	Gerencia General

PROYECTO 5: Mostrar columnas renombradas

Enunciado: Mostrar información de equipos con nombres amigables.

Código SQL:

```
SELECT
    Id_Inventario AS Codigo_Inventario,
    ID_Hardware AS Identificador_Hardware,
    ID_Software AS Identificador_Software
FROM Equipo_Inventario;
```

Análisis: Esta consulta mejora la presentación de la información al mostrar nombres de columnas más comprensibles para el usuario final.

Evidencia: Campos renombrados en el resultado de la consulta.

```
-- Proyecto 5 - Mostrar columnas renombradas
-- =====
-- Enunciado: Mostrar información de equipos con nombres amigables.
-- Verificación:
SELECT * FROM Equipo_Inventario;
-- Consulta del ejercicio:
SELECT
    Id_Inventario AS Codigo_Inventario,
    ID_Hardware AS Identificador_Hardware,
    ID_Software AS Identificador_Software
FROM Equipo_Inventario;
GO
```

121 %

Results Messages

	Id_Inventario	Id_CodigoAlmacen	ID_Hardware	ID_Software	ID_General	ID_Partes	Codigo_Anidesk
1	INV001	ALM001	HW001	SW001	GEN001	AP001	1001
2	INV002	ALM001	HW002	SW002	GEN002	AP002	1002
3	INV003	ALM001	HW003	SW003	GEN003	AP003	1003
4	INV004	ALM001	HW004	SW004	GEN004	AP004	1004
5	INV005	ALM001	HW005	SW005	GEN005	AP005	1005
6	INV006	ALM001	HW006	SW006	GEN006	AP006	1006
7	INV007	ALM001	HW007	SW007	GEN007	AP007	1007
8	INV008	ALM001	HW008	SW008	GEN008	AP008	1008

	Codigo_Inventario	Identificador_Hardware	Identificador_Software
1	INV001	HW001	SW001
2	INV002	HW002	SW002

Query executed successfully. DESKTOP-7VTBN0H\SQLEXPRESS ... DESKTOP

PROYECTO 6: Suma de memoria RAM

Enunciado: Sumar 4 GB adicionales a la memoria RAM mostrada.

Código SQL:

```
SELECT
    ID_Hardware,
    Ram AS RAM_Original,
    CAST(REPLACE(Ram, 'GB', '')) AS INT + 4 AS RAM_Calculada_Num,
    CONCAT(CAST(REPLACE(Ram, 'GB', '')) AS INT + 4, 'GB') AS RAM_Final_Actualizada
FROM Hardware;
```

Análisis: Se extrae el valor numérico de la RAM, se convierte a entero y luego se le suma 4. Esto demuestra el uso de funciones de texto, conversión de datos y operaciones aritméticas sobre valores almacenados como cadena.

Evidencia: Se muestra la RAM original y la RAM calculada con 4 GB adicionales.

```
-- =====
-- Proyecto 6 - Suma de memoria RAM
-- =====
-- Enunciado: Sumar 4 GB adicionales a la memoria RAM mostrada.
-- Verificación:
SELECT * FROM Hardware;
-- Consulta del ejercicio:
SELECT
    ID_Hardware,
    Ram AS RAM_Original,
    CAST(REPLACE(Ram, 'GB', '' ) AS INT) + 4 AS RAM_Calculada_Num,
    CONCAT(CAST(REPLACE(Ram, 'GB', '' ) AS INT) + 4, 'GB') AS RAM_Final_Actualizada
FROM Hardware;
```

121 %

Results Messages

	ID_Hardware	Codigo_Procesador	ID_DiscoDuro	Codigo_GPU	tipo	Marca_y_Modelo	N_Serie	Ram	Unidad_CD_DVDRW	Regulador
1	HW001	PROC001	DD001	GPU001	escritorio	Dell OptiPlex	12345	8GB	si	si
2	HW002	PROC002	DD002	GPU002	laptop	HP Pavilion	54321	16GB	no	si
3	HW003	PROC003	DD003	GPU003	all in one	Lenovo AIO	67890	4GB	si	no
4	HW004	PROC004	DD004	GPU004	escritorio	Acer Aspire	11223	8GB	no	si
5	HW005	PROC005	DD005	GPU005	laptop	Asus VivoBook	33445	12GB	si	si
6	HW006	PROC006	DD006	GPU006	escritorio	Intel NUC	55667	16GB	si	si
7	HW007	PROC007	DD007	GPU007	all in one	iMac	77889	8GB	no	no
8	HW008	PROC008	DD008	GPU008	laptop	MSI Gaming	99001	32GB	si	si

	ID_Hardware	RAM_Original	RAM_Calculada_Num	RAM_Final_Actualizada
1	HW001	8GB	12	12GB
2	HW002	16GB	20	20GB

Activar V

PROYECTO 7: Doble de la capacidad de disco

Enunciado: Calcular el doble de almacenamiento.

Código SQL:

```
SELECT
    ID_DiscoDuro,
    DiscoDuro_Capacidad AS Capacidad_Original,
    CASE
        WHEN DiscoDuro_Capacidad LIKE '%GB%' THEN
            CONCAT(CAST(LEFT(DiscoDuro_Capacidad, CHARINDEX('GB', DiscoDuro_Capacidad) -
1) AS INT) * 2, 'GB')
        WHEN DiscoDuro_Capacidad LIKE '%TB%' THEN
            CONCAT(CAST(LEFT(DiscoDuro_Capacidad, CHARINDEX('TB', DiscoDuro_Capacidad) -
1) AS INT) * 2, 'TB')
        ELSE 'No_Calculable'
    END AS Doble_Almacenamiento
FROM Disco_Duro;
```

Análisis: El ejercicio identifica si la capacidad está expresada en GB o TB, extrae el valor numérico y lo multiplica por 2. Se usa CASE para controlar diferentes unidades de almacenamiento.

Evidencia: Resultado con la capacidad original y el doble de almacenamiento calculado.

SQLQuery4.sql - DE...N0H\Pavilion (65))

```
-- =====
-- Proyecto 7 - Multiplicación de capacidad de disco
-- =====
-- Enunciado: Calcular el doble de almacenamiento.
-- Verificación:
SELECT * FROM Disco_Duro;
-- Consulta del ejercicio:
SELECT
    ID_DiscoDuro,
    DiscoDuro_Capacidad AS Capacidad_Original,
    CASE
        WHEN DiscoDuro_Capacidad LIKE '%GB%' THEN
            CONCAT(CAST(LEFT(DiscoDuro_Capacidad, CHARINDEX('GB', DiscoDuro_Capacidad) - 1) AS INT) * 2, 'GB')
    
```

121 %

Results Messages

ID_DiscoDuro	DiscoDuro_Capacidad	DiscoDuro_Marca
1	DD001	256GB SSD
2	DD002	512GB SSD
3	DD003	1TB HDD
4	DD004	2TB HDD
5	DD005	128GB SSD
6	DD006	256GB SSD
7	DD007	512GB SSD
8	DD008	1TB SSD

ID_DiscoDuro	Capacidad_Original	Doble_Almacenamiento
1	DD001	256GB SSD
2	DD002	512GB SSD

Activar Windows

Query executed successfully.

DESKTOP-7VTBNOH\SQLEXPRESS ... | DESKTOP-7VTBNOH\Pavili... | GestionInventarioTecnol... | 00:00:00

PROYECTO 8: División segura

Enunciado: Demostrar cómo evitar errores de división.

Código SQL:

```
DECLARE @Divisor INT = 0;
SELECT
    100 AS Numerador,
    @Divisor AS Divisor_Cero,
    100 / NULLIF(@Divisor, 0) AS Resultado_Division_Segura;
```

Análisis: NULLIF convierte el cero en NULL, evitando el error de división entre cero. Es una técnica útil para proteger cálculos matemáticos.

Evidencia: Consulta ejecutada sin generar error de división por cero.

The screenshot shows a SQL query window titled 'SQLQuery4.sql - DE...N0H\Pavilion (65))' with the following code:

```
-- =====
-- Proyecto 8 - División segura
-- =====
-- Enunciado: Demostrar cómo evitar errores de división.
-- Verificación: Operación en memoria.
DECLARE @Divisor INT = 0;
SELECT
    100 AS Numerador,
    @Divisor AS Divisor_Cero,
    100 / NULLIF(@Divisor, 0) AS Resultado_Division_Segura;
GO
```

Below the query window, the 'Results' tab is active, displaying a table with the following data:

	Numerador	Divisor_Cero	Resultado_Division_Segura
1	100	0	NULL

PROYECTO 9: Precedencia de operadores

Enunciado: Comparar operaciones matemáticas.

Código SQL:

```
SELECT
    5 + 3 * 2 AS Op_Sin_Parentesis,
    (5 + 3) * 2 AS Op_Con_Parentesis;
```

Análisis: Se observa cómo los paréntesis modifican el orden de las operaciones. Sin paréntesis, la multiplicación tiene prioridad.

Evidencia: Dos resultados distintos, según el uso o no de paréntesis.

The screenshot shows a SQL query window with the following code:

```
-- =====
-- Proyecto 9 - Uso de precedencia de operadores
-- =====
-- Enunciado: Comparar operaciones matemáticas.
-- Verificación: Operación en memoria.
SELECT
    5 + 3 * 2 AS Op_Sin_Parentesis,
    (5 + 3) * 2 AS Op_Con_Parentesis;
GO
```

PROYECTO 10: Manejo de valores nulos

Enunciado: Mostrar valores nulos reemplazados.

Código SQL:

```
SELECT
    ID_General,
    Estado,
    Observacion AS Observacion_Original,
    ISNULL(Observacion, 'Sin_Observaciones_Registradas') AS Observacion_Reemplazada
FROM General;
```

Análisis: ISNULL reemplaza valores vacíos por un texto definido, lo que evita que el resultado aparezca incompleto.

Evidencia: Las observaciones nulas son sustituidas por un texto alternativo.

The screenshot shows a SQL query window titled 'SQLQuery4.sql - DE...N0H\Pavilion (65))' with a 'GO' button. The query is as follows:

```
-- =====
-- Proyecto 10 - Manejo de valores nulos
-- =====
-- Enunciado: Mostrar valores nulos reemplazados.
-- Verificación:
SELECT * FROM General;
-- Consulta del ejercicio:
SELECT
    ID_General,
    Estado,
    Observacion AS Observacion_Original,
```

Below the query window, the 'Results' tab is active, displaying a table with 8 rows and 5 columns: ID_General, Estado, Observacion, Fima, and Observacion_Reemplazada. The 5th row (ID_General: GEN005) has a NULL value in the Observacion column, which is highlighted in yellow. The bottom part of the screenshot shows the 'Messages' tab with the text 'Query executed successfully.'

	ID_General	Estado	Observacion	Fima
1	GEN001	bueno	Equipo en excelente estado	si
2	GEN002	regular	Requiere limpieza de polvo	si
3	GEN003	malo	Disco duro fallando	si
4	GEN004	medio	Batería baja	si
5	GEN005	bueno	NULL	si
6	GEN006	regular	Teclado con teclas duras	si
7	GEN007	medio	Ventilador ruidoso	si
8	GEN008	bueno	Recién mantenido	si

	ID_General	Estado	Observacion_Original	Observacion_Reemplazada
1	GEN001	bueno	Equipo en excelente estado	Equipo en excelente estado
2	GEN002	regular	Requiere limpieza de polvo	Requiere limpieza de polvo

PROYECTO 11: Concatenar texto simple

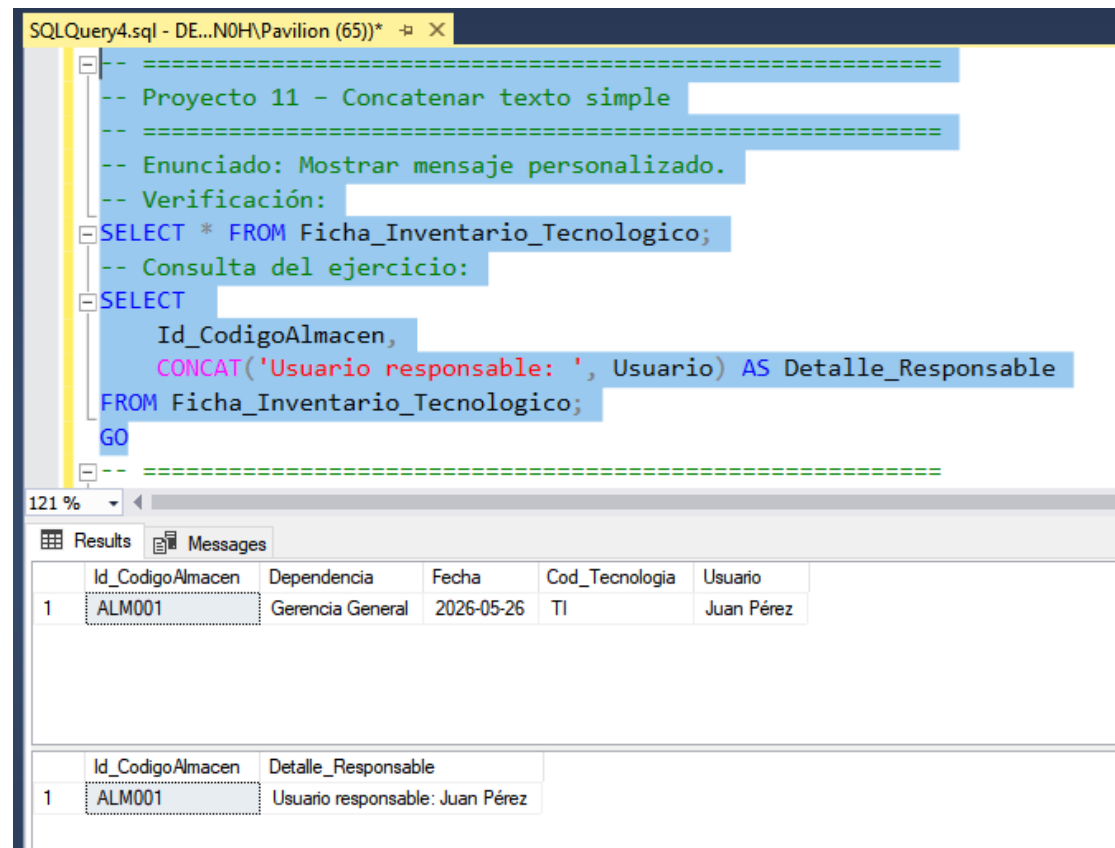
Enunciado: Mostrar mensaje personalizado.

Código SQL:

```
SELECT
    Id_CodigoAlmacen,
    CONCAT('Usuario responsable: ', Usuario) AS Detalle_Responsable
FROM Ficha_Inventario_Tecnologico;
```

Análisis: Se construye un mensaje legible combinando texto fijo con el nombre del usuario.

Evidencia: Columna con mensajes personalizados por registro.



The screenshot shows a SQL query window titled 'SQLQuery4.sql - DE...N0H\Pavilion (65))' with a query that concatenates a fixed string with the 'Usuario' field from the 'Ficha_Inventario_Tecnologico' table. Below the query window, the 'Results' tab displays two tables. The first table shows the original data with columns: Id_CodigoAlmacen, Dependencia, Fecha, Cod_Tecnologia, and Usuario. The second table shows the result of the query, with columns: Id_CodigoAlmacen and Detalle_Responsable, where the latter is the concatenated string.

	Id_CodigoAlmacen	Dependencia	Fecha	Cod_Tecnologia	Usuario
1	ALM001	Gerencia General	2026-05-26	TI	Juan Pérez

	Id_CodigoAlmacen	Detalle_Responsable
1	ALM001	Usuario responsable: Juan Pérez

PROYECTO 12: Concatenar marca y modelo

Enunciado: Unir marca y modelo.

Código SQL:

```
SELECT
    N_Serie_Monitor,
    CONCAT(marca, ' ', modelo) AS Especificacion_Monitor
FROM Monitor;
```

Análisis: La concatenación reúne dos campos para mostrar una descripción técnica más completa del monitor.

Evidencia: Se visualiza una sola columna con marca y modelo unidos.

SQLQuery4.sql - DE...N0H\Pavilion (65))*

```
-- =====
-- Proyecto 12 - Concatenar datos de equipo
-- =====
-- Enunciado: Unir marca y modelo.
-- Verificación:
SELECT * FROM Monitor;
-- Consulta del ejercicio:
SELECT
    N_Serie_Monitor,
    CONCAT(marca, ' ', modelo) AS Especificacion_Monitor
FROM Monitor;
GO
```

121 %

Results Messages

	N_Serie_Monitor	modelo	marca
1	MON001	24 pulgadas Full HD	LG
2	MON002	27 pulgadas 4K	Dell
3	MON003	22 pulgadas Full HD	HP
4	MON004	21.5 pulgadas HD	Asus
5	MON005	24 pulgadas Full HD	Samsung
6	MON006	23.8 pulgadas Full HD	BenQ
7	MON007	27 pulgadas QHD	LG
8	MON008	24 pulgadas IPS	Acer

	N_Serie_Monitor	Especificacion_Monitor
1	MON001	LG 24 pulgadas Full HD
2	MON002	Dell 27 pulgadas 4K

Query executed successfully. DESKTOP-7VTRN0H\SQL

PROYECTO 13: Concatenar con formato de texto

Enunciado: Mostrar información formateada.

Código SQL:

```
SELECT
    N_Serie_Mouse,
    CONCAT('Modelo: ', modelo, ' | N° Serie: ', N_Serie_Mouse) AS Detalles_Mouse
FROM Mouse;
```

Análisis: Se organiza la información en un formato más claro para reportes o impresión de datos.

Evidencia: Resultado con formato descriptivo del mouse.

SQLQuery4.sql - DE...N0H\Pavilion (65))

```
-- =====
-- Proyecto 13 - Concatenar con espacios
-- =====
-- Enunciado: Mostrar información formateada.
-- Verificación: (La tabla Mouse solo tiene Serie y Modelo según tu script)
SELECT * FROM Mouse;
-- Consulta del ejercicio:
SELECT
    N_Serie_Mouse,
    CONCAT('Modelo: ', modelo, ' | N° Serie: ', N_Serie_Mouse) AS Detalles_Mouse
FROM Mouse;
GO
```

121 %

Results Messages

	N_Serie_Mouse	modelo
1	MOU001	Mouse Genius
2	MOU002	Mouse Logitech M100
3	MOU003	Mouse HP
4	MOU004	Mouse Dell
5	MOU005	Mouse Microsoft
6	MOU006	Mouse Redragon
7	MOU007	Mouse Corsair
8	MOU008	Mouse Razer

	N_Serie_Mouse	Detalles_Mouse
1	MOU001	Modelo: Mouse Genius N° Serie: MOU001
2	MOU002	Modelo: Mouse Logitech M100 N° Serie: MOU002

PROYECTO 14: Concatenación con valores nulos

Enunciado: Evitar problemas de concatenación con NULL.

Código SQL:

```
SELECT
    ID_General,
    'Estado: ' + Estado + ' - Obs: ' + Observacion AS Falla_Operador_Suma,
    CONCAT('Estado: ', Estado, ' - Obs: ', Observacion) AS Concatenacion_Segura
FROM General;
```

Análisis: El uso del operador + puede fallar si alguno de los valores es NULL; en cambio, CONCAT maneja mejor estos casos.

Evidencia: Comparación entre concatenación tradicional y concatenación segura.

SQLQuery4.sql - DE...N0H\Pavilion (65))

```
-- =====
-- Proyecto 14 - Concatenación con valores nulos
-- =====
-- Enunciado: Evitar problemas de concatenación con NULL.
-- Verificación:
SELECT * FROM General;
-- Consulta del ejercicio:
SELECT
    ID_General,
    'Estado: ' + Estado + ' - Obs: ' + Observacion AS Falla_Operador_Suma,
    CONCAT('Estado: ', Estado, ' - Obs: ', Observacion) AS Concatenacion_Segura
FROM General;
```

121 %

Results Messages

	ID_General	Estado	Observacion	Firma
1	GEN001	bueno	Equipo en excelente estado	si
2	GEN002	regular	Requiere limpieza de polvo	si
3	GEN003	malo	Disco duro fallando	si
4	GEN004	medio	Batería baja	si
5	GEN005	bueno	NULL	si
6	GEN006	regular	Teclado con teclas duras	si
7	GEN007	medio	Ventilador ruidoso	si
8	GEN008	bueno	Recién mantenido	si

	ID_General	Falla_Operador_Suma	Concatenacion_Segura
1	GEN001	Estado: bueno - Obs: Equipo en excelente estado	Estado: bueno - Obs: Equipo en excelente estado
2	GEN002	Estado: regular - Obs: Requiere limpieza de polvo	Estado: regular - Obs: Requiere limpieza de polvo

Ac

PROYECTO 15: Uso de COALESCE

Enunciado: Mostrar valores alternativos.

Código SQL:

```
SELECT
    ID_General,
    Estado,
    COALESCE(Observacion, 'SIN_NOVEDADES') AS Estado_Observacion
FROM General;
```

Análisis: COALESCE devuelve el primer valor no nulo. En este caso, reemplaza observaciones vacías por un texto de respaldo.

Evidencia: Columna con valor alternativo cuando no existe observación.

SQLQuery4.sql - DE...N0H\Pavilion (65))*

```
-- =====
-- Proyecto 15 - Uso de COALESCE
-- =====
-- Enunciado: Mostrar valores alternativos.
-- Verificación:
SELECT * FROM General;
-- Consulta del ejercicio:
SELECT
    ID_General,
    Estado,
    COALESCE(Observacion, 'SIN_NOVEDADES') AS Estado_Observacion
FROM General;
GO
```

121 %

Results Messages

	ID_General	Estado	Observacion	Firma
1	GEN001	bueno	Equipo en excelente estado	si
2	GEN002	regular	Requiere limpieza de polvo	si
3	GEN003	malo	Disco duro fallando	si
4	GEN004	medio	Batería baja	si
5	GEN005	bueno	NULL	si
6	GEN006	regular	Teclado con teclas duras	si
7	GEN007	medio	Ventilador ruidoso	si
8	GEN008	bueno	Recién mantenido	si

	ID_General	Estado	Estado_Observacion
1	GEN001	bueno	Equipo en excelente estado
2	GEN002	regular	Requiere limpieza de polvo

Query executed successfully. DESKTOP-7VTBN0H\SOLEXPRESS

PROYECTO 16: Uso de igualdad

Enunciado: Mostrar equipos marca HP.

Código SQL:

```
SELECT N_Serie_Monitor, marca, modelo
FROM Monitor
WHERE marca = 'HP';
```

Análisis: El operador = permite filtrar registros que cumplen exactamente con la condición indicada.

Evidencia: Solo aparecen monitores de marca HP.

SQLQuery4.sql - DE...N0H\Pavilion (65))*

```
-- Proyecto 16 - Uso de igualdad
-- =====
-- Enunciado: Mostrar equipos marca HP.
-- Verificación:
-- SELECT * FROM Monitor;
-- Consulta del ejercicio:
-- SELECT N_Serie_Monitor, marca, modelo
-- FROM Monitor
-- WHERE marca = 'HP';
-- GO

-- =====
-- Proyecto 17 - Uso de desigualdad
```

121 %

Results Messages

	N_Serie_Monitor	modelo	marca
1	MON001	24 pulgadas Full HD	LG
2	MON002	27 pulgadas 4K	Dell
3	MON003	22 pulgadas Full HD	HP
4	MON004	21.5 pulgadas HD	Asus
5	MON005	24 pulgadas Full HD	Samsung
6	MON006	23.8 pulgadas Full HD	BenQ
7	MON007	27 pulgadas QHD	LG
8	MON008	24 pulgadas IPS	Acer

	N_Serie_Monitor	marca	modelo
1	MON003	HP	22 pulgadas Full HD
2	MON012	HP	26 pulgadas Full HD

PROYECTO 17: Uso de desigualdad

Enunciado: Mostrar equipos que no son HP.

Código SQL:

```
SELECT N_Serie_Monitor, marca, modelo
FROM Monitor
WHERE marca <> 'HP';
```

Análisis: El operador <> excluye registros que coinciden con el valor indicado.

Evidencia: Se obtienen monitores de marcas diferentes a HP.

SQLQuery4.sql - DE...N0H\Pavilion (65))*

```

GO
-- =====
-- Proyecto 17 - Uso de desigualdad
-- =====
-- Enunciado: Mostrar equipos que no son HP.
-- Verificación:
SELECT * FROM Monitor;
-- Consulta del ejercicio:
SELECT N_Serie_Monitor, marca, modelo
FROM Monitor
WHERE marca <> 'HP';
GO

```

121 %

Results Messages

	N_Serie_Monitor	modelo	marca
1	MON001	24 pulgadas Full HD	LG
2	MON002	27 pulgadas 4K	Dell
3	MON003	22 pulgadas Full HD	HP
4	MON004	21.5 pulgadas HD	Asus
5	MON005	24 pulgadas Full HD	Samsung
6	MON006	23.8 pulgadas Full HD	BenQ
7	MON007	27 pulgadas QHD	LG
8	MON008	24 pulgadas IPS	Acer

	N_Serie_Monitor	marca	modelo
1	MON001	LG	24 pulgadas Full HD
2	MON002	Dell	27 pulgadas 4K

Query executed successfully. DESKTOP-7VTRN0H\SC

PROYECTO 18: Uso de mayor que

Enunciado: Mostrar equipos con ID mayor a 10.

Código SQL:

```

SELECT Id_Inventario, ID_Hardware, ID_Software
FROM Equipo_Inventario
WHERE CAST(SUBSTRING(Id_Inventario, 4, LEN(Id_Inventario)) AS INT) > 10;

```

Análisis: Se extrae la parte numérica del identificador para comparar correctamente los valores. Esto es necesario porque el ID está almacenado como texto.

Evidencia: Inventarios con numeración superior a 10.

SQLQuery4.sql - DE...N0H\Pavilion (65))

```

GO

-- =====
-- Proyecto 18 - Uso de mayor que
-- =====
-- Enunciado: Mostrar equipos con id mayor a 10.
-- Verificación:
SELECT * FROM Equipo_Inventario;
-- Consulta del ejercicio:
SELECT Id_Inventario, ID_Hardware, ID_Software
FROM Equipo_Inventario
WHERE CAST(SUBSTRING(Id_Inventario, 4, LEN(Id_Inventario)) AS INT) > 10;
GO

```

121 %

Results Messages

	Id_Inventario	Id_CodigoAlmacen	ID_Hardware	ID_Software	ID_General	ID_Part	Codigo_Anidesk
1	INV001	ALM001	HW001	SW001	GEN001	AP001	1001
2	INV002	ALM001	HW002	SW002	GEN002	AP002	1002
3	INV003	ALM001	HW003	SW003	GEN003	AP003	1003
4	INV004	ALM001	HW004	SW004	GEN004	AP004	1004
5	INV005	ALM001	HW005	SW005	GEN005	AP005	1005
6	INV006	ALM001	HW006	SW006	GEN006	AP006	1006
7	INV007	ALM001	HW007	SW007	GEN007	AP007	1007
8	INV008	ALM001	HW008	SW008	GEN008	AP008	1008

	Id_Inventario	ID_Hardware	ID_Software
1	INV011	HW011	SW011
2	INV012	HW012	SW012

PROYECTO 19: Uso de AND

Enunciado: Mostrar laptops HP.

Código SQL:

```

SELECT E.Id_Inventario, H.tipo, M.marca
FROM Equipo_Inventario AS E
INNER JOIN Hardware AS H ON E.ID_Hardware = H.ID_Hardware
INNER JOIN Accesorio_partes AS AP ON E.ID_Part = AP.ID_Part
INNER JOIN Monitor AS M ON AP.N_Serie_Monitor = M.N_Serie_Monitor
WHERE H.tipo = 'laptop' AND M.marca = 'HP';

```

Análisis: Se combinan tablas relacionadas y se aplican dos condiciones al mismo tiempo. Solo se muestran registros que cumplen ambas.

Evidencia: Equipos que son laptops y además usan monitor HP.

SQLQuery4.sql - DE...N0H\Pavilion (65))

```
-- =====
-- Proyecto 19 - Uso de AND
-- =====
-- Enunciado: Mostrar laptops HP.
-- Verificación:
SELECT * FROM Equipo_Inventario;
SELECT * FROM Hardware;
SELECT * FROM Monitor;
-- Consulta del ejercicio:
SELECT E.Id_Inventario, H.tipo, M.marca
FROM Equipo_Inventario AS E
INNER JOIN Hardware AS H ON E.ID_Hardware = H.ID_Hardware
```

121 %

Results Messages

	Id_Inventario	Id_CodigoAlmacen	ID_Hardware	ID_Software	ID_General	ID_Partes	Codigo_Anidesk
1	INV001	ALM001	HW001	SW001	GEN001	AP001	1001
2	INV002	ALM001	HW002	SW002	GEN002	AP002	1002
3	INV003	ALM001	HW003	SW003	GEN003	AP003	1003
4	INV004	ALM001	HW004	SW004	GEN004	AP004	1004
5	INV005	ALM001	HW005	SW005	GEN005	AP005	1005
6	INV006	ALM001	HW006	SW006	GEN006	AP006	1006
7	INV007	ALM001	HW007	SW007	GEN007	AP007	1007
8	INV008	ALM001	HW008	SW008	GEN008	AP008	1008

	ID_Hardware	Codigo_Procesador	ID_DiscoDuro	Codigo_GPU	tipo	Marca_y_Modelo	N_Serie	Ram	Unidad_CD_DVDRW	Regulador
1	HW001	PROC001	DD001	GPU001	escritorio	Dell OptiPlex	12345	8GB	si	si
2	HW002	PROC002	DD002	GPU002	laptop	HP Pavilion	54321	16GB	no	si

PROYECTO 20: Uso de OR y paréntesis

Enunciado: Mostrar equipos HP o Dell tipo laptop.

Código SQL:

```
SELECT E.Id_Inventario, H.tipo, M.marca
FROM Equipo_Inventario AS E
INNER JOIN Hardware AS H ON E.ID_Hardware = H.ID_Hardware
INNER JOIN Accesorio_partes AS AP ON E.ID_Partes = AP.ID_Partes
INNER JOIN Monitor AS M ON AP.N_Serie_Monitor = M.N_Serie_Monitor
WHERE (M.marca = 'HP' OR M.marca = 'Dell')
AND H.tipo = 'laptop';
```

Análisis: Los paréntesis agrupan la condición de marca para que el motor evalúe primero las marcas permitidas y luego aplique el tipo de hardware.

Evidencia: Laptops de marca HP o Dell.

SQLQuery4.sql - DE...N0H\Pavilion (65))*

```
-- =====
-- Proyecto 20 - Uso de OR y paréntesis
-- =====
-- Enunciado: Mostrar equipos HP o Dell tipo laptop.
-- Verificación:
SELECT * FROM Equipo_Inventario;
SELECT * FROM Hardware;
SELECT * FROM Monitor;
-- Consulta del ejercicio:
SELECT E.Id_Inventario, H.tipo, M.marca
FROM Equipo_Inventario AS E
INNER JOIN Hardware AS H ON E.ID_Hardware = H.ID_Hardware
INNER JOIN Accesorio_partes AS AP ON E.ID_Part es = AP.ID_Part es
```

121 %

Results Messages

	Id_Inventario	Id_CodigoAlmacen	ID_Hardware	ID_Software	ID_General	ID_Part es	Codigo_Anidesk
1	INV001	ALM001	HW001	SW001	GEN001	AP001	1001
2	INV002	ALM001	HW002	SW002	GEN002	AP002	1002
3	INV003	ALM001	HW003	SW003	GEN003	AP003	1003
4	INV004	ALM001	HW004	SW004	GEN004	AP004	1004
5	INV005	ALM001	HW005	SW005	GEN005	AP005	1005
6	INV006	ALM001	HW006	SW006	GEN006	AP006	1006
7	INV007	ALM001	HW007	SW007	GEN007	AP007	1007
8	INV008	ALM001	HW008	SW008	GEN008	AP008	1008

	ID_Hardware	Codigo_Procesador	ID_DiscoDuro	Codigo_GPU	tipo	Marca_y_Modelo	N_Serie	Ram	Unidad_CD_DVDRW	Regulador
1	HW001	PROC001	DD001	GPU001	escritorio	Dell OptiPlex	12345	8GB	si	si
2	HW002	PROC002	DD002	GPU002	laptop	HP Pavilion	54321	16GB	no	no

PROYECTO 21: Uso de LIKE con porcentaje

Enunciado: Buscar usuarios que empiecen con U.

Código SQL:

```
SELECT Id_CodigoAlmacen, Usuario, Dependencia
FROM Ficha_Inventario_Tecnologico
WHERE Usuario LIKE 'U%';
```

Análisis: LIKE 'U%' busca valores que comiencen con la letra U. El símbolo % representa cualquier cantidad de caracteres.

Evidencia: Registros cuyo usuario inicia con U.

```
-- =====
-- Proyecto 21 - Uso de LIKE con porcentaje
-- =====
-- Enunciado: Buscar usuarios que empiecen con U.
-- Verificación:
SELECT * FROM Ficha_Inventario_Tecnologico;
-- Consulta del ejercicio:
SELECT Id_CodigoAlmacen, Usuario, Dependencia
FROM Ficha_Inventario_Tecnologico
WHERE Usuario LIKE 'U%';
GO
```

121 %

Results Messages

	Id_CodigoAlmacen	Dependencia	Fecha	Cod_Tecnologia	Usuario
1	ALM001	Gerencia General	2026-05-26	TI	Juan Pérez

Id_CodigoAlmacen	Usuario	Dependencia
------------------	---------	-------------

PROYECTO 22: Uso de LIKE con guion bajo

Enunciado: Buscar usuarios con patrón específico.

Código SQL:

```
SELECT Id_CodigoAlmacen, Usuario
FROM Ficha_Inventario_Tecnologico
WHERE Usuario LIKE '_a%';
```

Análisis: El guion bajo _ representa un solo carácter. Esto permite buscar patrones más precisos dentro del texto.

Evidencia: Usuarios que cumplen la estructura indicada en el patrón.

SQLQuery4.sql - DE...N0H\Pavilion (65))*

```
-- =====
-- Proyecto 22 - Uso de LIKE con guion bajo
-- =====
-- Enunciado: Buscar usuarios con patrón específico.
-- Verificación:
SELECT * FROM Ficha_Inventario_Tecnologico;
-- Consulta del ejercicio:
SELECT Id_CodigoAlmacen, Usuario
FROM Ficha_Inventario_Tecnologico
WHERE Usuario LIKE '_a%';
GO
-- =====
```

121 %

Results Messages

	Id_CodigoAlmacen	Dependencia	Fecha	Cod_Tecnologia	Usuario
1	ALM001	Gerencia General	2026-05-26	TI	Juan Pérez

Id_CodigoAlmacen Usuario

PROYECTO 23: Uso de BETWEEN

Enunciado: Mostrar equipos con IDs entre 5 y 15.

Código SQL:

```
SELECT Id_Inventario, ID_Hardware, ID_Software
FROM Equipo_Inventario
WHERE CAST(SUBSTRING(Id_Inventario, 4, LEN(Id_Inventario)) AS INT) BETWEEN 5 AND 15;
```

Análisis: BETWEEN permite filtrar rangos. Se convierte el identificador a entero para comparar correctamente los valores numéricos.

Evidencia: Inventarios comprendidos entre 5 y 15.

SQLQuery4.sql - DE...N0H\Pavilion (65))*

```
-- =====
-- Proyecto 23 - Uso de BETWEEN
-- =====
-- Enunciado: Mostrar equipos con IDs entre 5 y 15.
-- Verificación:
SELECT * FROM Equipo_Inventario;
-- Consulta del ejercicio:
SELECT Id_Inventario, ID_Hardware, ID_Software
FROM Equipo_Inventario
WHERE CAST(SUBSTRING(Id_Inventario, 4, LEN(Id_Inventario)) AS INT) BETWEEN 5 AND 15;
GO
```

121 %

Results Messages

	Id_Inventario	Id_CodigoAlmacen	ID_Hardware	ID_Software	ID_General	ID_Partes	Codigo_Anidesk
1	INV001	ALM001	HW001	SW001	GEN001	AP001	1001
2	INV002	ALM001	HW002	SW002	GEN002	AP002	1002
3	INV003	ALM001	HW003	SW003	GEN003	AP003	1003
4	INV004	ALM001	HW004	SW004	GEN004	AP004	1004
5	INV005	ALM001	HW005	SW005	GEN005	AP005	1005
6	INV006	ALM001	HW006	SW006	GEN006	AP006	1006
7	INV007	ALM001	HW007	SW007	GEN007	AP007	1007
8	INV008	ALM001	HW008	SW008	GEN008	AP008	1008

	Id_Inventario	ID_Hardware	ID_Software
1	INV005	HW005	SW005
2	INV006	HW006	SW006

Query executed successfully.

DESKTOP-7VTBN0H\SQLEXPRESS ... | DESKTOP-7VTBN0H\Pav

PROYECTO 24: Uso de IN con valores explícitos

Enunciado: Mostrar equipos HP y Dell.

Código SQL:

```
SELECT N_Serie_Monitor, marca, modelo
FROM Monitor
WHERE marca IN ('HP', 'Dell');
```

Análisis: IN simplifica la consulta cuando se desea verificar varios valores posibles en un mismo campo.

Evidencia: Monitores de marca HP y Dell.

SQLQuery4.sql - DE...N0H\Pavilion (65))*

```
-- =====
-- Proyecto 24 - Uso de IN con valores explícitos
-- =====
-- Enunciado: Mostrar equipos HP y Dell.
-- Verificación:
SELECT * FROM Monitor;
-- Consulta del ejercicio:
SELECT N_Serie_Monitor, marca, modelo
FROM Monitor
WHERE marca IN ('HP', 'Dell');
GO
```

121 %

Results Messages

	N_Serie_Monitor	modelo	marca
1	MON001	24 pulgadas Full HD	LG
2	MON002	27 pulgadas 4K	Dell
3	MON003	22 pulgadas Full HD	HP
4	MON004	21.5 pulgadas HD	Asus
5	MON005	24 pulgadas Full HD	Samsung
6	MON006	23.8 pulgadas Full HD	BenQ
7	MON007	27 pulgadas QHD	LG
8	MON008	24 pulgadas IPS	Acer

	N_Serie_Monitor	marca	modelo
1	MON002	Dell	27 pulgadas 4K
2	MON003	HP	22 pulgadas Full HD

PROYECTO 25: Uso de IN con subconsulta simple

Enunciado: Mostrar usuarios pertenecientes a dependencias existentes.

Código SQL:

```
SELECT Id_CodigoAlmacen, Usuario, Dependencia
FROM Ficha_Inventario_Tecnologico
WHERE Dependencia IN (
    SELECT DISTINCT Dependencia
    FROM Ficha_Inventario_Tecnologico
    WHERE Dependencia IS NOT NULL
);
```

Análisis: La subconsulta obtiene las dependencias válidas y la consulta principal filtra los registros que pertenecen a ese conjunto.

Evidencia: Usuarios asociados a dependencias registradas.

SQLQuery4.sql - DE...N0H\Pavilion (65))* -> X

```
-- =====
-- Proyecto 25 - Uso de IN con subconsulta simple
-- =====
-- Enunciado: Mostrar usuarios pertenecientes a dependencias existentes.
-- Verificación:
SELECT * FROM Ficha_Inventario_Tecnologico;
-- Consulta del ejercicio:
SELECT Id_CodigoAlmacen, Usuario, Dependencia
FROM Ficha_Inventario_Tecnologico
WHERE Dependencia IN (
    SELECT DISTINCT Dependencia
    FROM Ficha_Inventario_Tecnologico
    WHERE Dependencia IS NOT NULL
```

121 %

Results Messages

	Id_CodigoAlmacen	Dependencia	Fecha	Cod_Tecnologia	Usuario
1	ALM001	Gerencia General	2026-05-26	TI	Juan Pérez

	Id_CodigoAlmacen	Usuario	Dependencia
1	ALM001	Juan Pérez	Gerencia General

PROYECTO 26: Extraer año

Enunciado: Mostrar el año de registro.

Código SQL:

```
SELECT Id_CodigoAlmacen, Usuario, Fecha, YEAR(Fecha) AS Anio_Registro
FROM Ficha_Inventario_Tecnologico;
```

Análisis: La función YEAR() extrae únicamente el año de la fecha almacenada.

Evidencia: Se visualiza la fecha junto con el año extraído.

SQLQuery4.sql - DE...N0H\Pavilion (65))*

```
-- Proyecto 26 - Extraer año
-- =====
-- Enunciado: Mostrar el año de registro.
-- Verificación:
SELECT * FROM Ficha_Inventario_Tecnologico;
-- Consulta del ejercicio:
SELECT Id_CodigoAlmacen, Usuario, Fecha, YEAR(Fecha) AS Anio_Registro
FROM Ficha_Inventario_Tecnologico;
GO

-- =====
-- Proyecto 27 - Extraer mes y día
-- =====
```

121 %

Results Messages

	Id_CodigoAlmacen	Dependencia	Fecha	Cod_Tecnologia	Usuario
1	ALM001	Gerencia General	2026-05-26	TI	Juan Pérez

	Id_CodigoAlmacen	Usuario	Fecha	Anio_Registro
1	ALM001	Juan Pérez	2026-05-26	2026

PROYECTO 27: Extraer mes y día

Enunciado: Mostrar mes y día.

Código SQL:

```
SELECT
    Id_CodigoAlmacen,
    Fecha,
    MONTH(Fecha) AS Numero_Mes,
    DAY(Fecha) AS Dia_Registro
FROM Ficha_Inventario_Tecnologico;
```

Análisis: Las funciones MONTH() y DAY() permiten descomponer la fecha en partes específicas para análisis temporal.

Evidencia: Mes y día mostrados por separado.

SQLQuery4.sql - DE...N0H\Pavilion (65))*

```
-- Proyecto 27 - Extraer mes y día
-- =====
-- Enunciado: Mostrar mes y día.
-- Verificación:
-- SELECT * FROM Ficha_Inventario_Tecnologico;
-- Consulta del ejercicio:
-- SELECT
--     Id_CodigoAlmacen,
--     Fecha,
--     MONTH(Fecha) AS Numero_Mes,
--     DAY(Fecha) AS Dia_Registro
-- FROM Ficha_Inventario_Tecnologico;
GO
```

121 %

Results Messages

	Id_CodigoAlmacen	Dependencia	Fecha	Cod_Tecnologia	Usuario
1	ALM001	Gerencia General	2026-05-26	TI	Juan Pérez

	Id_CodigoAlmacen	Fecha	Numero_Mes	Dia_Registro
1	ALM001	2026-05-26	5	26

PROYECTO 28: Agregar días a una fecha

Enunciado: Calcular fecha futura.

Código SQL:

```
SELECT
    Id_CodigoAlmacen,
    Fecha AS Fecha_Revision_Original,
    DATEADD(DAY, 30, Fecha) AS Fecha_Proximo_Mantenimiento
FROM Ficha_Inventario_Tecnologico;
```

Análisis: DATEADD suma 30 días a la fecha original. Es útil para programar mantenimientos o revisiones posteriores.

Evidencia: Fecha original y fecha futura calculada.

SQLQuery4.sql - DE...N0H\Pavilion (65))*

```
-- Proyecto 28 - Agregar días a una fecha
-- =====
-- Enunciado: Calcular fecha futura.
-- Verificación:
-- SELECT * FROM Ficha_Inventario_Tecnologico;
-- Consulta del ejercicio:
-- SELECT
--     Id_CodigoAlmacen,
--     Fecha AS Fecha_Revision_Original,
--     DATEADD(DAY, 30, Fecha) AS Fecha_Proximo_Mantenimiento
-- FROM Ficha_Inventario_Tecnologico;
GO
```

121 %

Results Messages

	Id_CodigoAlmacen	Dependencia	Fecha	Cod_Tecnologia	Usuario
1	ALM001	Gerencia General	2026-05-26	TI	Juan Pérez

	Id_CodigoAlmacen	Fecha_Revision_Original	Fecha_Proximo_Mantenimiento
1	ALM001	2026-05-26	2026-06-25

PROYECTO 29: Diferencia entre fechas

Enunciado: Calcular días transcurridos.

Código SQL:

```
SELECT
    Id_CodigoAlmacen,
    Fecha AS Fecha_Ingreso,
    GETDATE() AS Fecha_Actual,
    DATEDIFF(DAY, Fecha, GETDATE()) AS Dias_Transcurridos
FROM Ficha_Inventario_Tecnologico;
```

Análisis: DATEDIFF calcula la diferencia entre la fecha registrada y la fecha actual, permitiendo medir tiempo transcurrido.

Evidencia: Cantidad de días transcurridos desde el registro.

SQLQuery4.sql - DE...N0H\Pavilion (65))*

```
-- =====
-- Proyecto 29 - Diferencia entre fechas
-- =====
-- Enunciado: Calcular días transcurridos.
-- Verificación:
SELECT * FROM Ficha_Inventario_Tecnologico;
-- Consulta del ejercicio:
SELECT
    Id_CodigoAlmacen,
    Fecha AS Fecha_Ingreso,
    GETDATE() AS Fecha_Actual,
    DATEDIFF(DAY, Fecha, GETDATE()) AS Dias_Transcurridos
FROM Ficha Inventario Tecnologico;
```

121 %

Results Messages

	Id_CodigoAlmacen	Dependencia	Fecha	Cod_Tecnologia	Usuario
1	ALM001	Gerencia General	2026-05-26	TI	Juan Pérez

	Id_CodigoAlmacen	Fecha_Ingreso	Fecha_Actual	Dias_Transcurridos
1	ALM001	2026-05-26	2026-05-26 19:47:49.610	0

PROYECTO 30: Convertir formato de fechas

Enunciado: Mostrar fechas en distintos formatos.

Código SQL:

```
SELECT
    Id_CodigoAlmacen,
    Fecha AS Formato_Nativo,
    CONVERT(VARCHAR, Fecha, 103) AS Formato_Latino_Corto,
    CONVERT(VARCHAR, Fecha, 111) AS Formato_Estandar_Japon
FROM Ficha_Inventario_Tecnologico;
```

Análisis: CONVERT permite presentar la fecha en distintos formatos según la necesidad del reporte o del usuario final.

Evidencia: Fecha mostrada en formato nativo, latino corto y estándar japonés.

```
-- Proyecto 30 - Convertir formato de fechas
-- =====
-- Enunciado: Mostrar fechas en distintos formatos.
-- Verificación:
-- SELECT * FROM Ficha_Inventario_Tecnologico;
-- Consulta del ejercicio:
-- SELECT
--     Id_CodigoAlmacen,
--     Fecha AS Formato_Nativo,
--     CONVERT(VARCHAR, Fecha, 103) AS Formato_Latino_Corto,
--     CONVERT(VARCHAR, Fecha, 111) AS Formato_Estandar_Japon
-- FROM Ficha_Inventario_Tecnologico;
GO
```

121 %

Results Messages

	Id_CodigoAlmacen	Dependencia	Fecha	Cod_Tecnologia	Usuario
1	ALM001	Gerencia General	2026-05-26	TI	Juan Pérez

	Id_CodigoAlmacen	Formato_Nativo	Formato_Latino_Corto	Formato_Estandar_Japon
1	ALM001	2026-05-26	26/05/2026	2026/05/26